Snapshotting is also an imperfect solution to within-bucket dependency problems, because it only handles read-only queries; locks are still required for inter-object mutations. The snapshots themselves can be expensive to update as well (although one easy-to-apply optimization here is to only generate snapshots for objects on an as-needed basis).

There are lots of other ways to tackle these problems. The best way to discover how to update game objects concurrently is to experiment. Hopefully we've presented some useful ideas in this section that will pave the way forward as you experiment for yourself!

16.8 Events and Message-Passing

Games are inherently event-driven. An *event* is anything of interest that happens during gameplay. An explosion going off, the player being sighted by an enemy, a health pack getting picked up—these are all events. Games generally need a way to (a) notify interested game objects when an event occurs and (b) arrange for those objects to respond to interesting events in various ways—we call this *handling* the event. Different types of game objects will respond in different ways to an event. The way in which a particular type of game object responds to an event is a crucial aspect of its behavior, just as important as how the object's state changes over time in the absence of any external inputs. For example, the behavior of the ball in *Pong* is governed in part by its velocity, in part by how it reacts to the event of striking a wall or paddle and bouncing off, and in part by what happens when the ball is missed by one of the players.

16.8.1 The Problem with Statically Typed Function Binding

One simple way to notify a game object that an event has occurred is to simply call a method (member function) on the object. For example, when an explosion goes off, we could query the game world for all objects within the explosion's damage radius and then call a virtual function named something like `OnExplosion()` on each one. This is illustrated by the following pseudocode:

```
void Explosion::Update()
{
    // ...

    if (ExplosionJustWentOff())
    {
        GameObjectCollection damagedObjects;
```

```
            g_world.QueryObjectsInSphere(GetDamageSphere(),
                                         damagedObjects);

            for (each object in damagedObjects)
            {
                object.OnExplosion(*this);
            }
        }

        // ...
    }
```

The call to `OnExplosion()` is an example of *statically typed late function binding*. Function binding is the process of determining which function implementation to invoke at a particular call location—the implementation is, in effect, bound to the call. Virtual functions, such as our `OnExplosion()` event-handling function, are said to be *late-bound*. This means that the compiler doesn't actually know *which* of the many possible implementations of the function is going to be invoked at compile time—only at runtime, when the type of the target object is known, will the appropriate implementation be invoked. We say that a virtual function call is *statically typed* because the compiler *does* know which implementation to invoke given a particular object type. It knows, for example, that `Tank::OnExplosion()` should be called when the target object is a `Tank` and that `Crate::OnExplosion()` should be called when the object is a `Crate`.

The problem with statically typed function binding is that it introduces a degree of inflexibility into our implementation. For one thing, the virtual `OnExplosion()` function requires all game objects to inherit from a common base class. Moreover, it requires that base class to *declare* the virtual function `OnExplosion()`, even if not all game objects can respond to explosions. In fact, using statically typed virtual functions as event handlers would require our base `GameObject` class to declare virtual functions for *all possible events* in the game! This would make adding new events to the system difficult. It precludes events from being created in a data-driven manner—for example, within the world editing tool. It also provides no mechanism for certain types of objects, or certain individual object instances, to register interest in certain events but not others. Every object in the game, in effect, "knows" about every possible event, even if its response to the event is to do nothing (i.e., to implement an empty, do-nothing event handler function).

What we really need for our event handlers, then, is *dynamically typed late function binding*. Some programming languages support this feature natively (e.g., C#'s delegates). In other languages, the engineers must implement it

manually. There are many ways to approach this problem, but most boil down to taking a data-driven approach. In other words, we encapsulate the notion of a function call in an object and pass that object around at runtime in order to implement a dynamically typed late-bound function call.

16.8.2 Encapsulating an Event in an Object

An event is really comprised of two components: its *type* (explosion, friend injured, player spotted, health pack picked up, etc.) and its *arguments*. The arguments provide specifics about the event. (How much damage did the explosion do? Which friend was injured? Where was the player spotted? How much health was in the health pack?) We can encapsulate these two components in an object, as shown by the following rather over-simplified code snippet:

```
struct Event
{
    const U32 MAX_ARGS = 8;

    EventType  m_type;
    U32        m_numArgs;
    EventArg   m_aArgs[MAX_ARGS];
};
```

Some game engines call these things *messages* or *commands* instead of *events*. These names emphasize the idea that informing objects about an event is essentially equivalent to sending a message or command to those objects.

Practically speaking, event objects are usually not quite this simple. We might implement different types of events by deriving them from a root event class, for example. The arguments might be implemented as a linked list or a dynamically allocated array capable of containing arbitrary numbers of arguments, and the arguments might be of various data types.

Encapsulating an event (or message) in an object has many benefits:

- *Single event handling function*. Because the event object encodes its type internally, any number of different event types can be represented by an instance of a single class (or the root class of an inheritance hierarchy). This means that we only need *one* virtual function to handle *all* types of events (e.g., `virtual void OnEvent(Event& event);`).
- *Persistence*. Unlike a function call, whose arguments go out of scope after the function returns, an event object stores both its type and its arguments as data. An event object therefore has persistence. It can be stored in a queue for handling at a later time, copied and broadcast to multiple receivers and so on.

- *Blind event forwarding*. An object can forward an event that it receives to another object without having to "know" anything about the event. For example, if a vehicle receives a Dismount event, it can forward it to all of its passengers, thereby allowing them to dismount the vehicle, even though the vehicle itself knows nothing about dismounting.

This idea of encapsulating an event/message/command in an object is commonplace in many fields of computer science. It is found not only in game engines but in other systems like graphical user interfaces, distributed communication systems and many others. The well-known "Gang of Four" design patterns book [19] calls this the Command design pattern.

16.8.3 Event Types

There are many ways to distinguish between different types of events. One simple approach in C or C++ is to define a global enum that maps each event type to a unique integer.

```
enum EventType
{
    EVENT_TYPE_LEVEL_STARTED,
    EVENT_TYPE_PLAYER_SPAWNED,
    EVENT_TYPE_ENEMY_SPOTTED,
    EVENT_TYPE_EXPLOSION,
    EVENT_TYPE_BULLET_HIT,
    // ...
}
```

This approach enjoys the benefits of simplicity and efficiency (since integers are usually extremely fast to read, write and compare). However, it also suffers from two problems. First, knowledge of *all* event types in the entire game is centralized, which can be seen as a form of broken encapsulation (for better or for worse—opinions on this vary). Second, the event types are hard-coded, which means new event types cannot easily be defined in a data-driven manner. Third, enumerators are just indices, so they are order-dependent. If someone accidentally adds a new event type in the middle of the list, the indices of all subsequent event ids change, which can cause problems if event ids are stored in data files. As such, an enumeration-based event typing system works well for small demos and prototypes but does not scale very well at all to real games.

Another way to encode event types is via strings. This approach is totally free-form, and it allows a new event type to be added to the system by merely thinking up a name for it. But it suffers from many problems, including a

strong potential for event name conflicts, the possibility of events not working because of a simple typo, increased memory requirements for the strings themselves, and the relatively high cost of comparing strings relative to that of comparing integers. Hashed string ids can be used instead of raw strings to eliminate the performance problems and increased memory requirements, but they do nothing to address event name conflicts or typos. Nonetheless, the extreme flexibility and data-driven nature of a string- or string-id-based event system is considered worth the risks by many game teams, including Naughty Dog.

Tools can be implemented to help avoid some of the risks involved in using strings to identify events. For example, a central database of all event type names could be maintained. A user interface could be provided to permit new event types to be added to the database. Naming conflicts could be automatically detected when a new event is added, and the user could be disallowed from adding duplicate event types. When selecting a preexisting event, the tool could provide a sorted list in a drop-down combo box rather than requiring the user to remember the name and type it manually. The event database could also store metadata about each type of event, including documentation about its purpose and proper usage and information about the number and types of arguments it supports. This approach can work really well, but we should not forget to account for the costs of setting up and maintaining such a system, as they are not insignificant.

16.8.4 Event Arguments

The arguments of an event usually act like the argument list of a function, providing information about the event that might be useful to the receiver. Event arguments can be implemented in all sorts of ways.

We might derive a new type of `Event` class for each unique type of event. The arguments can then be hard-coded as data members of the class. For example:

```
class ExplosionEvent : public Event
{
    Point  m_center;
    float  m_damage;
    float  m_radius;
};
```

Another approach is to store the event's arguments as a collection of *variants*. A variant is a data object that is capable of holding more than one type of data. It usually stores information about the data type that is currently being stored, as well as the data itself. In an event system, we might want our argu-

ments to be integers, floating-point values, Booleans or hashed string ids. So in C or C++, we could define a variant class that looks something like this:

```
struct Variant
{
    enum Type
    {
        TYPE_INTEGER,
        TYPE_FLOAT,
        TYPE_BOOL,
        TYPE_STRING_ID,
        TYPE_COUNT // number of unique types
    };

    Type        m_type;

    union
    {
        I32     m_asInteger;
        F32     m_asFloat;
        bool    m_asBool;
        U32     m_asStringId;
    };
};
```

The collection of variants within an `Event` might be implemented as an array with a small, fixed maximum size (say 4, 8 or 16 elements). This imposes an arbitrary limit on the number of arguments that can be passed with an event, but it also side-steps the problems of dynamically allocating memory for each event's argument payload, which can be a big benefit, especially in memory-constrained console games.

The collection of variants might be implemented as a dynamically sized data structure, like a dynamically sized array (like `std::vector`) or a linked list (like `std::list`). This provides a great deal of additional flexibility over a fixed size design, but it incurs the cost of dynamic memory allocation. A pool allocator could be used to great effect here, presuming that each `Variant` is the same size.

16.8.4.1 Event Arguments as Key-Value Pairs

A fundamental problem with an *indexed* collection of event arguments is order dependency. Both the sender and the receiver of an event must "know" that the arguments are listed in a specific order. This can lead to confusion and bugs. For example, a required argument might be accidentally omitted or an extra one added.

This problem can be avoided by implementing event arguments as key-value pairs. Each argument is uniquely identified by its key, so the arguments can appear in any order, and optional arguments can be omitted altogether. The argument collection might be implemented as a closed or open hash table, with the keys used to hash into the table, or it might be an array, linked list or binary search tree of key-value pairs. These ideas are illustrated in Table 16.1. The possibilities are numerous, and the specific choice of implementation is largely unimportant as long as the game's particular requirements have been effectively and efficiently met.

16.8.5 Event Handlers

When an event, message or command is received by a game object, it needs to respond to the event in some way. This is known as *handling* the event, and it is usually implemented by a function or a snippet of script code called an *event handler*. (We'll have more to learn about game scripting later on.)

Often an event handler is a single native virtual function or script function that is capable of handling all types of events (e.g., `virtual void OnEvent(Event& event)`). In this case, the function usually contains some kind of switch statement or cascaded if/else-if clause to handle the various types of events that might be received. A typical event handler function might look something like this:

```
virtual void SomeObject::OnEvent(Event& event)
{
    switch (event.GetType())
    {
    case SID("EVENT_ATTACK"):
        RespondToAttack(event.GetAttackInfo());
        break;

    case SID("EVENT_HEALTH_PACK"):
```

Key	Value	
	Type	
"event"	stringid	"explosion"
"radius"	float	10.3
"damage"	int	25
"grenade"	bool	true

Table 16.1. The arguments of an event object can be implemented as a collection of key-value pairs. The keys prevent order-dependency problems because each event argument is uniquely identified by its key.

```
            AddHealth(event.GetHealthPack().GetHealth());
            break;

        // ...

        default:
            // Unrecognized event.
            break;
        }
    }
```

Alternatively, we might implement a suite of handler functions, one for each type of event (e.g., `OnThis()`, `OnThat()`, …). However, as we discussed above, a proliferation of event handler functions can be problematic.

A Windows GUI toolkit called Microsoft Foundation Classes (MFC) was well-known for its *message maps*—a system that permitted any Windows message to be bound at runtime to an arbitrary non-virtual or virtual function. This avoided the need to declare handlers for all possible Windows messages in a single root class, while at the same time avoiding the big switch statement that is commonplace in non-MFC Windows message-handling functions. But such a system is probably not worth the hassle—a switch statement works really well and is simple and clear.

16.8.6 Unpacking an Event's Arguments

The example above glosses over one important detail—namely, how to extract data from the event's argument list in a type-safe manner. For example, `event.GetHealthPack()` presumably returns a `HealthPack` game object, which in turn we presume provides a member function called `GetHealth()`. This implies that the root `Event` class "knows" about health packs (as well as, by extension, every other type of event argument in the game). This is probably an impractical design. In a real engine, there might be derived `Event` classes that provide convenient data-access APIs such as `GetHealthPack()`. Or the event handler might have to unpack the data manually and cast them to the appropriate types. This latter approach raises type safety concerns, although practically speaking it usually isn't a huge problem because the type of the event is always known when the arguments are unpacked.

16.8.7 Chains of Responsibility

Game objects are almost always dependent upon one another in various ways. For example, game objects usually reside in a transformation hierarchy, which allows an object to rest on another object or be held in the hand of a character. Game objects might also be made up of multiple interacting components,

leading to a star topology or a loosely connected "cloud" of component objects. A sports game might maintain a list of all the characters on each team. In general, we can envision the interrelationships between game objects as one or more *relationship graphs* (remembering that a list and a tree are just special cases of a graph). A few examples of relationship graphs are shown in Figure 16.21.

It often makes sense to be able to pass events from one object to the next within these relationship graphs. For example, when a vehicle receives an event, it may be convenient to pass the event to all of the passengers riding on the vehicle, and those passengers may wish to forward the event to the objects in their inventories. When a multicomponent game object receives an event, it may be necessary to pass the event to all of the components so that they all get a crack at handling it. Or when an event is received by a character in a sports game, we might want to pass it on to all of his or her teammates as well.

The technique of forwarding events within a graph of objects is a common design pattern in object-oriented, event-driven programming, sometimes referred to as *Chain of Responsibility* [19]. Usually, the order in which the event is passed around the system is predetermined by the engineers. The event is passed to the first object in the chain, and the event handler returns a Boolean or an enumerated code indicating whether or not it recognized and handled the event. If the event is consumed by a receiver, the process of event forwarding stops; otherwise, the event is forwarded on to the next receiver in the chain. An event handler that supports Chain of Responsibility style event forwarding might look something like this:

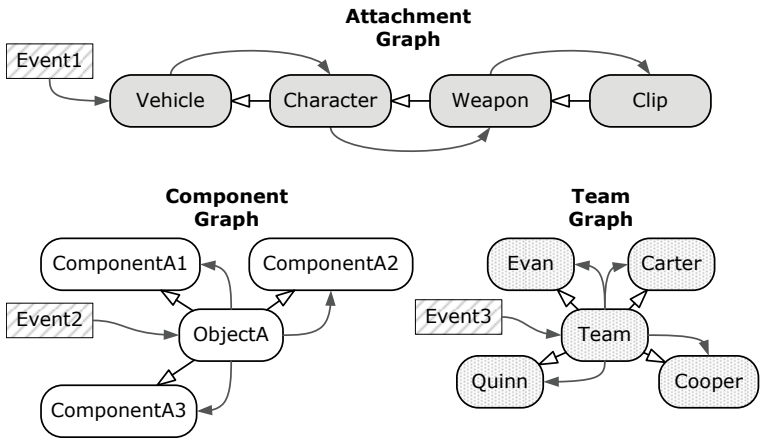

Figure 16.21. Game objects are interrelated in various ways, and we can draw graphs depicting these relationships. Any such graph might serve as a distribution channel for events.

```
virtual bool SomeObject::OnEvent(Event& event)
{
    // Call the base class' handler first.
    if (BaseClass::OnEvent(event))
    {
        return true;
    }

    // Now try to handle the event myself.
    switch (event.GetType())
    {
    case SID("EVENT_ATTACK"):
        RespondToAttack(event.GetAttackInfo());
        return false; // OK to forward this event to others.

    case SID("EVENT_HEALTH_PACK"):
        AddHealth(event.GetHealthPack().GetHealth());
        return true; // I consumed the event; don't forward.

    // ...

    default:
        return false; // I didn't recognize this event.
    }
}
```

When a derived class overrides an event handler, it can be appropriate to call the base class's implementation as well if the class is augmenting but not replacing the base class's response. In other situations, the derived class might be entirely replacing the response of the base class, in which case the base class's handler should not be called. This is another kind of responsibility chain.

Event forwarding has other applications as well. For example, we might want to multicast an event to all objects within a radius of influence (for an explosion, for example). To implement this, we can leverage our game world's object query mechanism to find all objects within the relevant sphere and then forward the event to all of the returned objects.

16.8.8 Registering Interest in Events

It's reasonably safe to say that most objects in a game do not need to respond to every possible event. Most types of game objects have a relatively small set of events in which they are "interested." This can lead to inefficiencies when multicasting or broadcasting events, because we need to iterate over a group

of objects and call each one's event handler, even if the object is not interested in that particular kind of event.

One way to overcome this inefficiency is to permit game objects to *register interest* in particular kinds of events. For example, we could maintain one linked list of interested game objects for each distinct type of event, or each game object could maintain a bit array, in which the setting of each bit corresponds to whether or not the object is interested in a particular type of event. By doing this, we can avoid calling the event handlers of any objects that do not care about the event.

Even better, we might be able to restrict our original game object query to include only those objects that are interested in the event we wish to multicast. For example, when an explosion goes off, we can ask the collision system for all objects that are within the damage radius *and* that can respond to Explosion events. This can save time overall, because we avoid iterating over objects that we know aren't interested in the event we're multicasting. Whether or not such an approach will produce a net gain depends on how the query mechanism is implemented and the relative costs of filtering the objects during the query versus filtering them during the multicast iteration.

16.8.9 To Queue or Not to Queue

Most game engines provide a mechanism for handling events immediately when they are sent. In addition to this, some engines also permit events to be queued for handling at an arbitrary future time. Event queuing has some attractive benefits, but it also increases the complexity of the event system significantly and poses some unique problems. We'll investigate the pros and cons of event queuing in the following sections and learn how such systems are implemented in the process.

16.8.9.1 Some Benefits of Event Queuing

The following sections outline some of the benefits of event queuing.

Control over When Events Are Handled

We have seen that we must be careful to update engine subsystems and game objects in a specific order to ensure correct behavior and maximize runtime performance. In the same sense, certain kinds of events may be highly sensitive to exactly when within the game loop they are handled. If all events are handled immediately upon being sent, the event handler functions end up being called in unpredictable and difficult-to-control ways throughout the course of the game loop. By deferring events via an event queue, the engi-

neers can take steps to ensure that events are only handled when it is safe and appropriate to do so.

Ability to Post Events into the Future

When an event is sent, the sender can usually specify a delivery time—for example, we might want the event to be handled later in the same frame, next frame or some number of seconds after it was sent. This feature amounts to an ability to post events into the future, and it has all sorts of interesting uses. We can implement a simple alarm clock by posting an event into the future. A periodic task, such as blinking a light every two seconds, can be executed by posting an event whose handler performs the periodic task and then posts a new event of the same type one time period into the future.

To implement the ability to post events into the future, each event is stamped with a desired delivery time prior to being queued. An event is only handled when the current game clock matches or exceeds its delivery time. An easy way to make this work is to sort the events in the queue in order of increasing delivery time. Each frame, the first event on the queue can be inspected and its delivery time checked. If the delivery time is in the future, we abort immediately because we know that all subsequent events are also in the future. But if we see an event whose delivery time is now or in the past, we extract it from the queue and handle it. This continues until an event is found whose delivery time is in the future. The following pseudocode illustrates this process:

```
// This function is called at least once per frame. Its
// job is to dispatch all events whose delivery time is
// now or in the past.

void EventQueue::DispatchEvents(F32 currentTime)
{
    // Look at, but don't remove, the next event on the
    // queue.
    Event* pEvent = PeekNextEvent();

    while (pEvent
        && pEvent->GetDeliveryTime() <= currentTime)
    {
        // Remove the event from the queue.
        RemoveNextEvent();

        // Dispatch it to its receiver's event handler.
        pEvent->Dispatch();

        // Peek at the next event on the queue (again
```

```
            // without removing it).
            pEvent = PeekNextEvent();
        }
    }
```

Event Prioritization

Even if our events are sorted by delivery time in the event queue, the order of delivery is still ambiguous when two or more events have exactly the same delivery time. This can happen more often than you might think, because it is quite common for events' delivery times to be quantized to an integral number of frames. For example, if two senders request that events be dispatched "this frame," "next frame" or "in seven frames from now," then those events will have identical delivery times.

One way to resolve these ambiguities is to assign *priorities* to events. Whenever two events have the same time stamp, the one with higher priority should always be serviced first. This is easily accomplished by first sorting the event queue by increasing delivery times and then sorting each group of events with identical delivery times in order of decreasing priority.

We could allow up to four billion unique priority levels by encoding our priorities in a raw, unsigned 32-bit integer, or we could limit ourselves to only two or three unique priority levels (e.g., low, medium and high). In every game engine, there exists some minimum number of priority levels that will resolve all real ambiguities in the system. It's usually best to aim as close to this minimum as possible. With a very large number of priority levels, it can become a small nightmare to figure out which event will be handled first in any given situation. However, the needs of every game's event system are different, and your mileage may vary.

16.8.9.2 Some Problems with Event Queuing

Increased Event System Complexity

In order to implement a queued event system, we need more code, additional data structures and more complex algorithms than would be necessary to implement an immediate event system. Increased complexity usually translates into longer development times and a higher cost to maintain and evolve the system during development of the game.

Deep-Copying Events and Their Arguments

With an immediate event handling approach, the data in an event's arguments need only persist for the duration of the event handling function (and any

functions it may call). This means that the event and its argument data can reside literally anywhere in memory, including on the call stack. For example, we could write a function that looks something like this:

```
void SendExplosionEventToObject(GameObject& receiver)
{
    // Allocate event args on the call stack.
    Point centerPoint(-2.0f, 31.5f, 10.0f);
    F32   damage = 5.0f;
    F32   radius = 2.0f;

    // Allocate the event on the call stack.
    Event event("Explosion");
    event.SetArgFloat("Damage", damage);
    event.SetArgPoint("Center", &centerPoint);
    event.SetArgFloat("Radius", radius);

    // Send the event, which causes the receiver's event
    // handler to be called immediately, as shown below.
    event.Send(receiver);
    //{
    //    receiver.OnEvent(event);
    //}
}
```

When an event is queued, its arguments must persist beyond the scope of the sending function. This implies that we must copy the entire event object prior to storing the event in the queue. We must perform a *deep-copy*, meaning that we copy not only the event object itself but its entire argument payload as well, including any data to which it may be pointing. Deep-copying the event ensures that there are no dangling references to data that exist only in the sending function's scope, and it permits the event to be stored indefinitely. The example event-sending function shown above still looks basically the same when using a queued event system, but as you can see in the italicized code below, the implementation of the `Event::Queue()` function is a bit more complex than its `Send()` counterpart:

```
void SendExplosionEventToObject(GameObject& receiver)
{
    // We can still allocate event args on the call
    // stack.
    Point centerPoint(-2.0f, 31.5f, 10.0f);
    F32   damage = 5.0f;
    F32   radius = 2.0f;
```

```
        // Still OK to allocate the event on the call stack
        Event event("Explosion
        event.SetArgFloat("Damage", damage);
        event.SetArgPoint("Center", &centerPoint);
        event.SetArgFloat("Radius", radius);

        // This stores the event in the receiver's queue for
        // handling at a future time. Note how the event
        // must be deep-copied prior to being enqueued, since
        // the original event resides on the call stack and
        // will go out of scope when this function returns.
        event.Queue(receiver);
        //{
        //    Event* pEventCopy = DeepCopy(event);
        //    receiver.EnqueueEvent(pEventCopy);
        //}
    }
```

Dynamic Memory Allocation for Queued Events

Deep-copying of event objects implies a need for dynamic memory allocation, and as we've already noted many times, dynamic allocation is undesirable in a game engine due to its potential cost and its tendency to fragment memory. Nonetheless, if we want to queue events, we'll need to dynamically allocate memory for them.

As with all dynamic allocation in a game engine, it's best if we can select a fast and fragmentation-free allocator. We might be able to use a *pool allocator*, but this will only work if all of our event objects are the same size and if their argument lists are comprised of data elements that are themselves all the same size. This may well be the case—for example, the arguments might each be a `Variant`, as described above. If our event objects and/or their arguments can vary in size, a *small memory allocator* might be applicable. (Recall that a small memory allocator maintains multiple pools, one for each of a few predetermined small allocation sizes.) When designing a queued event system, always be careful to take dynamic allocation requirements into account.

Other designs are possible, of course. For example, at Naughty Dog we allocate queued events as relocatable memory blocks. See Section 6.2.2.2 for more information on relocatable memory.

Debugging Difficulties

With queued events, the event handler is not called directly by the sender of that event. So, unlike in immediate event handling, the call stack does not tell us where the event came from. We cannot walk up the call stack in the debugger to inspect the state of the sender or the circumstances under which the event was sent. This can make debugging deferred events a bit tricky, and

things get even more difficult when events are forwarded from one object to another.

Some engines store debugging information that forms a paper trail of the event's travels throughout the system, but no matter how you slice it, event debugging is usually much easier in the absence of queuing.

Event queuing also leads to interesting and hard-to-track-down *race condition* bugs. We may need to pepper multiple event dispatches throughout our game loop, to ensure that events are delivered without incurring unwanted one-frame delays yet still ensuring that game objects are updated in the proper order during the frame. For example, during the animation update, we might detect that a particular animation has run to completion. This might cause an event to be sent whose handler wants to play a new animation. Clearly, we want to avoid a one-frame delay between the end of the first animation and the start of the next. To make this work, we need to update animation clocks first (so that the end of the animation can be detected and the event sent); then we should dispatch events (so that the event handler has a chance to request a new animation), and finally we can start animation blending (so that the first frame of the new animation can be processed and displayed). This is illustrated in the code snippet below:

```
while (true) // main game loop
{
    // ...

    // Update animation clocks. This may detect the end
    // of a clip, and cause EndOfAnimation events to
    // be sent.
    g_animationEngine.UpdateLocalClocks(dt);

    // Next, dispatch events. This allows an
    // EndOfAnimation event handler to start up a new
    // animation this frame if desired.
    g_eventSystem.DispatchEvents();

    // Finally, start blending all currently playing
    // animations (including any new clips started
    // earlier this frame).
    g_animationEngine.StartAnimationBlending();

    // ...
}
```

16.8.10 Some Problems with Immediate Event Sending

Not queuing events also has its share of issues. For example, immediate event handling can lead to extremely deep call stacks. Object A might send object B an event, and in its event handler, B might send another event, which might send another event, and another and so on. In a game engine that supports immediate event handling, it's not uncommon to see a call stack that looks something like this:

```
...
ShoulderAngel::OnEvent()
Event::Send()
Characer::OnEvent()
Event::Send()
Car::OnEvent()
Event::Send()
HandleSoundEffect()
AnimationEngine::PlayAnimation()
Event::Send()
Character::OnEvent()
Event::Send()
Character::OnEvent()
Event::Send()
Character::OnEvent()
Event::Send()
Car::OnEvent()
Event::Send()
Car::OnEvent()
Event::Send()
Car::Update()
GameWorld::UpdateObjectsInBucket()
Engine::GameLoop()
main()
```

A deep call stack like this can exhaust available stack space in extreme cases (especially if we have an infinite loop of event sending), but the real crux of the problem here is that every event handler function must be written to be fully *re-entrant*. This means that the event handler can be called recursively without any ill side-effects. As a contrived example, imagine a function that increments the value of a global variable. If the global is supposed to be incremented only once per frame, then this function is *not* re-entrant, because multiple recursive calls to the function will increment the variable multiple times.

16.8.11 Data-Driven Event/Message-Passing Systems

Event systems give the game programmer a great deal of flexibility over and above what can be accomplished with the statically typed function calling mechanisms provided by languages like C and C++. However, we can do better. In our discussions thus far, the logic for sending and receiving events is still hard-coded and therefore under the exclusive control of the engineers. If we could make our event system data-driven, we could extend its power into the hands of our game designers.

There are many ways to make an event system data-driven. Starting with the extreme of an entirely hard-coded (non-data-driven) event system, we could imagine providing some simple data-driven configurability. For example, designers might be allowed to configure how individual objects, or entire classes of object, respond to certain events. In the world editor, we can imagine selecting an object and then bringing up a scrolling list of all possible events that it might receive. For each one, the designer could use drop-down combo boxes and check boxes to control if, and how, the object responds, by selecting from a set of hard-coded, predefined choices. For example, given the event "PlayerSpotted," AI-controlled characters might be configured to do one of the following actions: run away, attack or ignore the event altogether. The event systems of some real commercial game engines are implemented in essentially this way.

At the other end of the gamut, our engine might provide the game designers with a simple scripting language (a topic we'll explore in detail in Section 16.9). In this case, the designer can literally write code that defines how a particular kind of game object will respond to a particular kind of event. In a scripted model, the designers are really just programmers (working with a somewhat less powerful but also easier-to-use and hopefully less error-prone language than the engineers), so anything is possible. Designers might define new types of events, send events and receive and handle events in arbitrary ways. This is what we do at Naughty Dog.

The problem with a simple, configurable event system is that it can severely limit what the game designers are capable of doing on their own, without the help of a programmer. On the other hand, a fully scripted solution has its own share of problems: Many game designers are not professional software engineers by training, so some designers find learning and using a scripting language a daunting task. Designers are also probably more prone to introducing bugs into the game than their engineer counterparts, unless they have practiced scripting or programming for some time. This can lead to some nasty surprises during alpha.

As a result, some game engines aim for a middle ground. They employ sophisticated graphical user interfaces to provide a great deal of flexibility without going so far as to provide users with a full-fledged, free-form scripting language. One approach is to provide a flowchart-style graphical programming language. The idea behind such a system is to provide the user with a limited and controlled set of atomic operations from which to choose but with plenty of freedom to wire them up in arbitrary ways. For example, in response to an event like "PlayerSpotted," the designer could wire up a flowchart that causes a character to retreat to the nearest cover point, play an animation, wait 5 seconds, and then attack. A GUI can also provide error-checking and validation to help ensure that bugs aren't inadvertently introduced. Unreal's Blueprints is an example of such a system—see the following section for more details.

16.8.11.1 Data Pathway Communication Systems

One of the problems with converting a function-call-like event system into a data-driven system is that different types of events tend to be incompatible. For example, let's imagine a game in which the player has an electromagnetic pulse gun. This pulse causes lights and electronic devices to turn off, scares small animals and produces a shock wave that causes any nearby plants to sway. Each of these game object types may already have an event response that performs the desired behavior. A small animal might respond to the "Scare" event by scurrying away. An electronic device might respond to the "TurnOff" event by turning itself off. And plants might have an event handler for a "Wind" event that causes them to sway. The problem is that our EMP gun is not compatible with any of these objects' event handlers. As a result, we end up having to implement a new event type, perhaps called "EMP," and then write custom event handlers for every type of game object in order to respond to it.

One solution to this problem is to take the event type out of the equation and to think solely in terms of *sending streams of data* from one game object to another. In such a system, every game object has one or more *input ports* to which a data stream can be connected, and one or more output ports through which data can be sent to other objects. Provided we have some way of wiring these ports together, such as a graphical user interface in which ports can be connected to each other via rubber-band lines, then we can construct arbitrarily complex behaviors. Continuing our example, the EMP gun would have an output port, perhaps named "Fire," that sends a Boolean signal. Most of the time, the port produces the value 0 (false), but when the gun is fired, it sends a brief (one-frame) pulse of the value 1 (true). The other game objects in

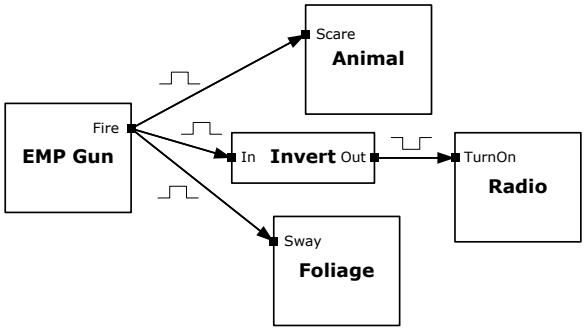

Figure 16.22. The EMP gun produces a 1 at its "Fire" output when fired. This can be connected to any input port that expects a Boolean value, in order to trigger the behavior associated with that input.

the world have binary input ports that trigger various responses. The animals might have a "Scare" input, the electronic devices a "TurnOn" input and the foliage objects a "Sway" input. If we connect the EMP gun's "Fire" output port to the input ports of these game objects, we can cause the gun to trigger the desired behaviors. (Note that we'd have to pipe the gun's "Fire" output through a node that *inverts* its input, prior to connecting it to the "TurnOn" input of the electronic devices. This is because we want them to turn *off* when the gun is firing.) The wiring diagram for this example is shown in Figure 16.22.

Programmers decide what kinds of port(s) each type of game object will have. Designers using the GUI can then wire these ports together in arbitrary ways in order to construct arbitrary behaviors in the game. The programmers also provide various other kinds of nodes for use within the graph, such as a node that inverts its input, a node that produces a sine wave or a node that outputs the current game time in seconds.

Various types of data might be sent along a data pathway. Some ports might produce or expect Boolean data, while others might be coded to produce or expect data in the form of a unit float. Still others might operate on 3D vectors, colors, integers and so on. It's important in such a system to ensure that connections are only made between ports with compatible data types, or we must provide some mechanism for automatically converting data types when two differently typed ports are connected together. For example, connecting a unit-float output to a Boolean input might automatically cause any value less than 0.5 to be converted to false, and any value greater than or equal to 0.5 to be converted to true. This is the essence of GUI-based event systems like Unreal Engine 4's Blueprints. A screenshot of Blueprints is shown in Figure 16.23.